



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Asignatura: Programación Orientada a Objetos

Proyecto: Sistema de Gestión y Reservación de Vuelos en Java

Reporte Técnico Final: Análisis, Diseño e Implementación del Sistema “Aeroviajes”

Equipo de Desarrollo:

- Francisco José Coutiño Morales (Núm. Cta: 425121623)
- Ernesto Flamenco Villaseñor (Núm. Cta: 319170599)

29 de mayo de 2026

Índice

1. Introducción	2
2. Objetivos del Proyecto	2
3. Flujo de Trabajo y Metodología de Desarrollo	2
4. Arquitectura del Sistema	3
5. Análisis de los Conceptos POO Implementados	4
5.1. Uso de Paquetes y Estructura Modular	4
5.2. Modificadores de Acceso	5
5.3. Herencia y Polimorfismo	5
5.4. Clases Abstractas e Interfaces	6
5.5. Colecciones	6
5.6. Manejo de Excepciones	7
5.7. Manejo de Archivos	7
5.8. Patrones de Diseño	7
6. Desarrollo del Proyecto	8
7. Detección y Corrección de Errores de Integración	8
8. Diagrama de Clases Final (As-Built)	8
9. Conclusiones Individuales	9
9.1. Francisco José Coutiño Morales	9
9.2. Ernesto Flamenco Villaseñor	10

1. Introducción

El presente documento detalla la arquitectura, las decisiones de diseño y el flujo de desarrollo del proyecto **Aeroviajes**, un sistema de gestión y reservación de vuelos codificado íntegramente en Java sobre el IDE Apache NetBeans 21. El sistema integra los paradigmas de la Programación Orientada a Objetos (POO) [1], el *Java Collections Framework* [2], persistencia mediante serialización de objetos, programación concurrente con hilos y la aplicación de seis patrones de diseño clásicos del catálogo GoF [3].

El objetivo fundamental del proyecto fue modelar un escenario comercial real —el de una aerolínea que debe gestionar usuarios, vuelos, reservas y boletos— y construir un sistema funcional completo con interfaz gráfica de usuario en Java Swing, control de acceso por roles (administrador y cliente) y persistencia transaccional. A diferencia de un sistema de consola, Aeroviajes implementa una arquitectura por capas que separa claramente la presentación, la lógica de negocio, la persistencia y el dominio, demostrando así un nivel de madurez arquitectónica apto tanto para evaluación académica como para servir como pieza de portafolio profesional.

2. Objetivos del Proyecto

Objetivo general: Desarrollar un sistema de reservación de vuelos en Java aplicando todos los conceptos del curso de Programación Orientada a Objetos, integrando además patrones de diseño, concurrencia y una interfaz gráfica funcional.

Objetivos específicos:

- Diseñar una arquitectura por capas con separación estricta de responsabilidades.
- Implementar un esquema de control de acceso diferenciado entre administradores y clientes, validado en tiempo de ejecución.
- Aplicar los seis patrones de diseño GoF que cubran las tres categorías clásicas (creacionales, estructurales y conductuales).
- Integrar programación concurrente mediante hilos y estructuras *thread-safe*, acopladas a la lógica de negocio mediante el patrón Productor-Consumidor.
- Persistir todos los datos del sistema (usuarios, vuelos, reservas) mediante serialización de objetos para garantizar continuidad entre ejecuciones.
- Producir una interfaz gráfica navegable, intuitiva y profesional.

3. Flujo de Trabajo y Metodología de Desarrollo

El desarrollo siguió un enfoque iterativo e incremental, con repartición vertical del trabajo entre los dos integrantes del equipo. La metodología se dividió en cuatro fases principales:

1. **Fase de diseño conceptual:** Se definieron las entidades del dominio, los paquetes lógicos y los patrones de diseño aplicables. Se elaboró un diagrama UML preliminar con PlantUML, un README profesional para el repositorio GitHub y un cronograma de trabajo dividido por días y por responsable. El repositorio se configuró como repositorio público en GitHub con el nombre `Sistema-de-Gestion-y-Reservacion-de-Vuelos`, bajo licencia MIT.
2. **Fase de codificación en paralelo:** El equipo dividió la responsabilidad por capas verticales para minimizar las dependencias cruzadas. Francisco se encargó del paquete `model` (entidades del dominio), `persistence` (repositorios y manejo de archivos), los patrones Singleton, Factory y Flyweight, el paquete `util` y la GUI de autenticación y administración. Ernesto se encargó del paquete `exception`, el paquete `service` (gestores de negocio), los patrones Strategy, Observer y Proxy, el paquete `threads` y la GUI del flujo cliente. Esta división permitió que ambos avanzaran sin bloquearse mutuamente, ya que las únicas dependencias críticas eran `model` y `exception`, que se construyeron primero.
3. **Fase de integración:** Una vez que ambos integrantes subieron sus paquetes al repositorio compartido, se procedió a integrar las piezas. Durante esta fase se detectaron incompatibilidades entre las firmas de métodos esperadas por cada lado (descritas en la Sección 7), las cuales se resolvieron mediante ajustes en las clases del modelo y los gestores, sin requerir reescrituras profundas. La `VentanaPrincipal` fue refactorizada para instanciar y coordinar todos los servicios, repositorios e hilos del sistema.
4. **Fase de pruebas funcionales y documentación:** Se ejecutó un *smoke test* manual del flujo completo (registro → login → consulta → compra → ticket → cierre → reinicio → verificación de persistencia) para validar que todas las capas funcionaran end-to-end. Se redactaron el reporte técnico, el manual de usuario y se actualizó el diagrama UML al estado *As-Built*.

4. Arquitectura del Sistema

El sistema sigue una **arquitectura por capas** con dependencias unidireccionales descendentes:

```

+-----+
| Capa de Presentacion      . aeroviajes.ui          | <- Swing (
JFrame, JPanel)
+-----+
| Capa de Servicio          . aeroviajes.service     | <- Logica de
negocio
+-----+
| Capa de Persistencia      . aeroviajes.persistence | <- Archivos /
serializacion
+-----+
| Capa de Dominio          . aeroviajes.model        | <- Entidades
del negocio
+-----+

```

```
^ patterns . threads . exception . util (transversales)
```

La presentación depende del servicio, el servicio del dominio y de la persistencia, pero **nunca al revés**. Las capas transversales (patrones, hilos, excepciones, utilidades) son consumidas por todas las anteriores sin acoplarlas entre sí. Esta separación facilita el mantenimiento, las pruebas y la extensibilidad del sistema.

5. Análisis de los Conceptos POO Implementados

A continuación se detalla cómo se aplicó cada uno de los conceptos requeridos por la rúbrica, indicando ubicación específica en el código y la justificación de diseño detrás de cada decisión.

5.1. Uso de Paquetes y Estructura Modular

El proyecto está organizado en **ocho paquetes lógicos** que reflejan la arquitectura por capas. Esta separación no es decorativa: cada paquete encapsula una responsabilidad única y oculta sus detalles internos del resto del sistema. La estructura es la siguiente:

Paquete	Responsabilidad
<code>aeroviajes.model</code>	Entidades del dominio del negocio (Usuario, Vuelo, Reserva, Ticket, Aeropuerto, Aerolínea).
<code>aeroviajes.service</code>	Lógica de negocio (gestores de usuarios, vuelos, reservas y tickets).
<code>aeroviajes.persistence</code>	Repositorios y manejo de archivos (serialización a <code>.dat</code> y escritura de tickets <code>.txt</code>).
<code>aeroviajes.exception</code>	Excepciones personalizadas del dominio.
<code>aeroviajes.patterns</code>	Implementaciones de los seis patrones GoF, divididas en seis sub-paquetes.
<code>aeroviajes.threads</code>	Hilos y la cola <i>thread-safe</i> del patrón Productor-Consumidor.
<code>aeroviajes.ui</code>	Interfaz gráfica de usuario (JFrame y paneles Swing).
<code>aeroviajes.util</code>	Utilidades transversales (generadores, validadores, constantes).

El paquete `patterns` se subdivide internamente en `singleton`, `factory`, `flyweight`, `proxy`, `strategy` y `observer`, agrupando cada patrón en su propia carpeta. Esta organización facilita identificar dónde vive cada patrón y explicarlo en una exposición.

5.2. Modificadores de Acceso

El sistema aplica los cuatro niveles de acceso de Java de forma consciente y consistente, encapsulando el estado interno de las entidades y exponiendo únicamente las operaciones necesarias.

- **private:** atributos de todas las entidades del modelo. La contraseña del `Usuario` es un ejemplo crítico: se almacena como `private` y **no se expone mediante `getter`**, forzando al resto del sistema a usar el método `validarContrasena(String intento)` para verificarla. Esto previene fugas accidentales de credenciales.
- **protected:** usado en la clase abstracta `Usuario` para que sus subclases (`Administrador` y `Cliente`) puedan acceder a los atributos heredados sin exponerlos al exterior. Su constructor es también `protected` para impedir instanciación directa.
- **public:** reservado para la API que cada clase ofrece al sistema (constructores, métodos de negocio, getters).
- **Package-private (sin modificador):** utilizado en clases internas de implementación que no deben ser visibles fuera de su paquete.

Adicionalmente, los atributos inmutables (como el `id` de un `Vuelo`, los códigos IATA de un `Aeropuerto` o el `codigoTicket`) se declaran como `private final`, garantizando que no pueden alterarse tras la construcción del objeto. Las clases que implementan el patrón Flyweight (`Aeropuerto`, `Aerolinea`) están declaradas como `public final`, impidiendo su extensión y reforzando su inmutabilidad.

5.3. Herencia y Polimorfismo

El sistema demuestra herencia y polimorfismo en cinco jerarquías distintas, todas con propósito funcional real:

1. **Jerarquía de usuarios:** La clase abstracta `Usuario` define los atributos comunes y el método abstracto `getRol()`. Sus dos subclases concretas, `Administrador` y `Cliente`, sobrescriben este método. Adicionalmente, `Cliente` agrega un historial de reservas mediante una colección `List<Reserva>`. Esto habilita el flujo polimórfico en el login: tras autenticar al usuario, el sistema invoca `u.getRol()` sin conocer la subclase concreta.
2. **Implementaciones de `IRepositorio<T>`:** Tres clases (`RepositorioUsuarios`, `RepositorioVuelos`, `RepositorioReservas`) heredan de la clase abstracta genérica `RepositorioBase<T>`. Cada subclase reutiliza toda la lógica CRUD definida en la base (principio DRY).
3. **Implementaciones de `IGestorVuelos`:** Dos clases (`GestorVuelosReal` y `GestorVuelosProxy`) implementan la misma interfaz. El proxy envuelve al real e intercepta las operaciones de modificación para validar permisos.
4. **Implementaciones de `IValidadorPago`:** `ValidadorTarjetaCredito` y `ValidadorTarjetaDebito` implementan la misma interfaz, intercambiables en tiempo de ejecución (patrón Strategy).

5. **Jerarquía de hilos:** `HiloConsumidorTickets` extiende `Thread`, `HiloAutoGuardado` implementa `Runnable` y `HiloCargaVuelos` extiende `SwingWorker`. Cada uno demuestra una forma distinta de heredar el comportamiento concurrente de Java.

5.4. Clases Abstractas e Interfaces

El sistema implementa dos clases abstractas y cinco interfaces:

Clases abstractas:

- **Usuario** (en `model`): define la estructura común pero no puede instanciarse directamente. Obliga a implementar `getRol()`.
- **RepositorioBase<T>** (en `persistence`): centraliza la lógica CRUD genérica y define el método abstracto `obtenerId(T entidad)`. Este es el patrón **Template Method**.

Interfaces:

- **IRepositorio<T>**: contrato genérico de los repositorios.
- **IGestorVuelos**: contrato del gestor de vuelos.
- **IValidadorPago**: contrato de los validadores de pago.
- **IObservadorReserva**: contrato de los observadores para notificaciones.
- **PanelActualizable**: permite callbacks `alMostrarse()` en paneles Swing.

5.5. Colecciones

El proyecto utiliza intensivamente el Java Collections Framework:

- **ArrayList<T>**: en `RepositorioBase` para mantener la colección de entidades en memoria. Elegido por su acceso secuencial eficiente.
- **HashMap<String, Usuario>**: en `GestorUsuarios` para indexar los usuarios por su correo electrónico en tiempo constante $O(1)$ [5].
- **HashMap<String, Vuelo>**: en `GestorVuelosReal` por la misma razón.
- **HashMap<String, Aeropuerto>** y **HashMap<String, Aerolinea>**: en las fábricas `Flyweight`, cachean las instancias.
- **BlockingQueue<Reserva>**: cola *thread-safe* que sirve de puente entre el hilo de la GUI y el hilo trabajador del patrón Productor-Consumidor [6].
- **List<T> como tipo de retorno**: los repositorios devuelven copias defensivas de la interfaz para impedir que el llamador modifique la colección interna.

5.6. Manejo de Excepciones

El sistema define **seis excepciones personalizadas** que heredan de `Exception` (verificadas):

Excepción	Cuándo se lanza
<code>VueloNoDisponible</code>	Al intentar acceder a un vuelo cuyo id no existe.
<code>AsientoNoDisponible</code>	Al intentar reservar un asiento en un vuelo lleno.
<code>UsuarioNoEncontrado</code>	Al buscar un usuario que no existe en el sistema.
<code>UsuarioYaExiste</code>	Al intentar registrar un correo ya tomado.
<code>Autenticacion</code>	Cuando una operación está protegida o una contraseña es incorrecta.
<code>PagoInvalido</code>	Cuando la validación de un método de pago falla.

5.7. Manejo de Archivos

La persistencia se basa en la **serialización nativa de Java** (`Serializable`). La capa está estructurada en dos niveles:

- **Nivel bajo (GestorArchivos):** Clase de utilidad estática que encapsula `ObjectOutputStream` y `ObjectInputStream`.
- **Nivel alto (Repositorios):** `cargar()` se invoca automáticamente en el constructor, y `persistir()` se llama tras cada operación de modificación.

5.8. Patrones de Diseño

El sistema implementa **seis patrones GoF** [3]:

- **Singleton (SistemaAeroviajes):** garantiza una única instancia coordinadora.
- **Factory Method (UsuarioFactory):** centraliza la creación de subtipos de `Usuario`.
- **Flyweight (AeropuertoFlyweightFactory):** optimiza memoria compartiendo las instancias de aeropuertos y aerolíneas.
- **Proxy (GestorVuelosProxy):** aplica un *Protection Proxy* para validar permisos de administrador.
- **Strategy (IValidadorPago):** encapsula algoritmos de validación en clases intercambiables.
- **Observer (NotificadorReservas):** notifica eventos a los observadores suscritos desacoplando la lógica.
- **Patrón de concurrencia: Productor-Consumidor.**

6. Desarrollo del Proyecto

La implementación del sistema se construyó capa por capa, de abajo hacia arriba:

Capa de dominio. Se comenzó por el paquete `model`, definiendo las ocho entidades centrales.

Capa de persistencia. Se diseñó la interfaz `IRepositorio<T>` y la clase `RepositorioBase<T>`.

Capa de excepciones. Se definieron las excepciones verificadas.

Capa de servicio. Se construyeron los cuatro gestores (`GestorUsuarios`, `GestorVuelosReal`, `GestorReservas`, `GestorTickets`).

Capa de patrones. Se desarrollaron los seis sub-paquetes arquitectónicos.

Capa de hilos. Implementación de `ColaTickets` y los tres procesos en segundo plano.

Capa de presentación. Interfaz en Swing con un único `JFrame` y patrón `CardLayout`.

7. Detección y Corrección de Errores de Integración

Al fusionar los paquetes desarrollados en paralelo, surgieron incompatibilidades que fueron corregidas:

1. **Discrepancia en constructores:** Se ajustaron las llamadas en `GestorUsuarios` para generar el ID automáticamente mediante `UUID`.
2. **Acceso a la contraseña:** Se refactorizó el gestor para usar `u.validarContrasena(input)` respetando el encapsulamiento.
3. **Nombres de getters:** Se homogeneizaron las llamadas desde la UI al modelo.
4. **Falta de persistencia en `GestorVuelosReal`:** Se solucionó cargando y guardando desde `RepositorioVuelos` en cada mutación.
5. **Refactorización de `PanelLogin`:** Se simplificó la navegación al panel de administrador una vez que este fue desarrollado.

8. Diagrama de Clases Final (As-Built)

A continuación se muestra el diagrama UML de clases con la arquitectura definitiva del sistema, incluyendo todos los paquetes, las relaciones de herencia, implementación de interfaces, composición y dependencia.

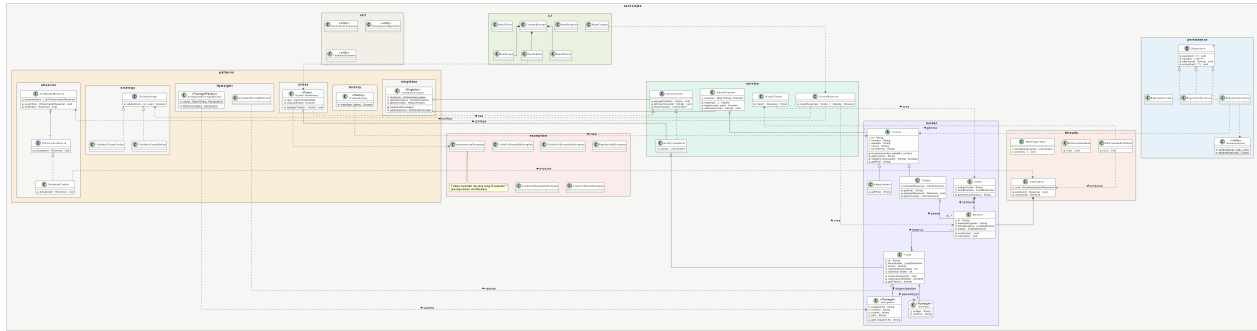


Figura 1: Diagrama de clases final (As-Built) del sistema Aerovias.

El diagrama agrupa visualmente los ocho paquetes mediante colores distintos: **util** y **ui** (presentación y soporte), **patterns** con sus seis sub-paquetes (singleton, strategy, flyweight, proxy, factory, observer), **service** (lógica de negocio), **exception** (excepciones del dominio), **model** (entidades), **persistence** (repositorios) y **threads** (hilos). Las relaciones tipadas muestran cómo se conectan: herencia para la jerarquía **Usuario** → **Administrador/Cliente**, implementación de interfaces para los repositorios y validadores, agregación entre **Cliente** y sus reservas, y composición fuerte entre **Ticket** y **Reserva**. Las flechas con estereotipos («crea», «notifica», «protege», «cachea», «produce», «consume») expresan la dirección y el rol de cada dependencia, comunicando la dinámica del sistema más allá de la pura estructura.

9. Conclusiones Individuales

9.1. Francisco José Coutiño Morales

El desarrollo de Aerovias representó para mí un salto respecto a los proyectos y practicas anteriores del curso. Diseñamos desde el inicio una arquitectura por capas con ocho paquetes lógicos, interfaz gráfica completa, persistencia transaccional y seis patrones de diseño cuidadosamente seleccionados.

Lo más difisil pero importante del proceso fue entender que **los patrones de diseño que expusimos en la clase y el como implementarlos en el proyecto, abarcando todos los temas posibles del curso**, y como estos son respuestas concretas a problemas reales: el Flyweight, por ejemplo, no se justifica "porque está en GoF", se justifica porque cien vuelos comparten diez aeropuertos y duplicar esos datos sería un desperdicio de memoria.

La aplicación del Template Method en **RepositorioBase<T>** fue uno de los momentos más satisfactorios del proyecto: ver cómo trescientas líneas de lógica CRUD se centralizan en una clase abstracta y los repositorios concretos quedan reducidos a quince líneas cada uno es la mejor demostración del principio DRY que se puede pedir. El proyecto también puso a prueba habilidades que no son estrictamente técnicas: coordinación con Ernesto sobre firmas de métodos antes de programar en paralelo y la resolución de problemas bajo presión de tiempo, que fue lo que mas nos costo y por lo que nos sirvio mucho los dias extra que nos dio para la entrega del proyecto.

9.2. Ernesto Flamenco Villaseñor

Trabajar en el Sistema de Gestion y Reservacion de Vuelos - Aeroviajes me permitió consolidar conceptos que en cursos anteriores había aplicado de forma aislada. La parte que más disfruté fue diseñar el flujo del patrón Productor-Consumidor acoplado al Observer: cuando vi por primera vez en la consola el mensaje Reserva en cola”seguido del consumidor procesándola en un hilo separado, entendí en concreto qué significa la concurrencia útil.

Implementar el Proxy fue particularmente revelador: la elegancia de poder interceptar cada operación crítica y validar el rol del usuario sin tocar la implementación real demostró el concepto y alcance que tiene el polimorfismo. El trabajo en equipo tuvo sus retos, pero me enseñó que la coordinación previa es una necesidad cuando dos personas trabajan en paralelo. Considero que cumplimos con los objetivos planteados al inicio del proyecto.

Bibliografía

- [1] Oracle. (s.f.). *Lesson: Object-Oriented Programming Concepts*. The Java Tutorials. Recuperado de <https://docs.oracle.com/javase/tutorial/java/concepts/index.html>
- [2] Oracle. (s.f.). *Lesson: Introduction to Collections*. The Java Tutorials. Recuperado de <https://docs.oracle.com/javase/tutorial/collections/intro/index.html>
- [3] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [4] Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., & Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley.
- [5] Oracle. (s.f.). *Class HashMap*. Java Platform, Standard Edition 21 API Specification. Recuperado de <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/HashMap.html>
- [6] Oracle. (s.f.). *Interface BlockingQueue*. Java Platform, Standard Edition 21 API Specification. Recuperado de <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/BlockingQueue.html>
- [7] Bell, D., & Parr, M. (2003). *Java para estudiantes* (3^a ed.). Pearson Educación.